

CSS GRID

1. Introdução

O **CSS Grid Layout**, normalmente chamado apenas de **CSS Grid**, é um sistema de layout do CSS utilizado para organizar elementos em **linhas e colunas**.

Com Grid podemos criar estruturas como:

- páginas;
- galerias;
- conjuntos de cards;
- dashboards;
- menus;
- formulários;
- áreas de conteúdo;
- layouts completos.

Uma das principais características do Grid é permitir o controle simultâneo de **linhas e colunas**.

Podemos visualizar um Grid como uma tabela:

	COLUNA 1	COLUNA 2	COLUNA 3
LINHA 1	[A]	[B]	[C]
LINHA 2	[D]	[E]	[F]
LINHA 3	[G]	[H]	[I]

Cada elemento ocupa uma ou mais células do Grid.

2. Conceito fundamental: Grid Container e Grid Items

Para trabalhar com CSS Grid precisamos compreender dois conceitos.

Grid Container

É o elemento que recebe:

```
display: grid;
```

Esse elemento passa a ser o **container do Grid**.

Grid Items

São os elementos que estão diretamente dentro do Grid Container.

Exemplo:

```
<div class="container">

  <div>Item 1</div>
  <div>Item 2</div>
  <div>Item 3</div>

</div>
.container {
  display: grid;
}
```

Nesse exemplo:

```
  .container
    ↓
  Grid Container

      div
      div
      div
      ↓
  Grid Items
```

3. Ativando o CSS Grid

A propriedade principal é:

```
display: grid;
```

Exemplo:

```
.container {
  display: grid;
}
```

HTML:

```
<div class="container">
  <div>Item 1</div>
  <div>Item 2</div>
  <div>Item 3</div>
  <div>Item 4</div>
</div>
```

Quando utilizamos `display: grid`, o elemento passa a utilizar o sistema de Grid.

Por padrão, os elementos serão organizados de acordo com as regras automáticas do Grid.

4. Criando colunas com grid-template-columns

A propriedade:

`grid-template-columns`
define a estrutura das colunas.

Exemplo:

```
.container {  
  display: grid;  
  
  /* Cria três colunas */  
  grid-template-columns: 200px 200px 200px;  
}
```

Temos:

```
+-----+-----+-----+  
|      |      |      |  
|   1   |   2   |   3   |  
|      |      |      |  
+-----+-----+-----+
```

Cada coluna possui 200px.

5. Criando colunas com porcentagem

Também podemos utilizar %.

```
.container {  
  display: grid;  
  
  /* Três colunas, cada uma com 33,33% */  
  grid-template-columns: 33.33% 33.33% 33.33%;  
}
```

Porém, existem formas mais práticas de dividir o espaço utilizando a unidade fr.

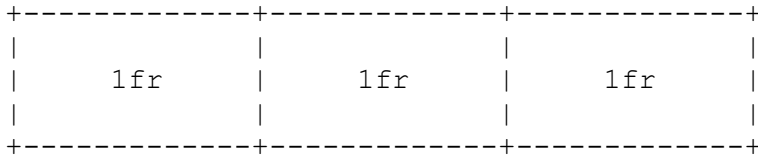
6. A unidade fr

A unidade fr significa **fraction**, ou seja, fração do espaço disponível.

Exemplo:

```
.container {  
  display: grid;  
  
  /* Divide o espaço em três partes iguais */  
  grid-template-columns: 1fr 1fr 1fr;  
}
```

Visualmente:



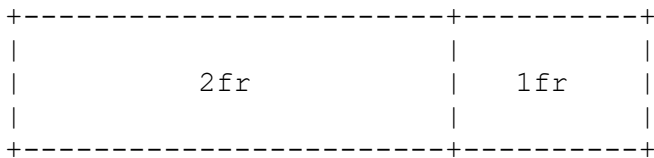
Cada coluna recebe uma fração igual do espaço disponível.

7. Diferentes proporções com fr

Podemos utilizar valores diferentes.

```
.container {  
  display: grid;  
  
  /* Primeira coluna recebe 2 partes */  
  /* Segunda coluna recebe 1 parte */  
  
  grid-template-columns: 2fr 1fr;  
}
```

O espaço será dividido em três partes:



A primeira coluna possui o dobro do espaço da segunda.

8. Exemplo prático com fr

HTML:

```
<div class="container">  
  
  <div class="item">Item 1</div>  
  <div class="item">Item 2</div>  
  
</div>
```

CSS:

```
.container {  
  display: grid;  
  
  /* Primeira coluna: 2 partes */  
  /* Segunda coluna: 1 parte */  
  grid-template-columns: 2fr 1fr;  
  
  /* Espaço entre os elementos */  
  gap: 20px;  
}
```

```
.item {
  padding: 30px;
  background-color: lightgray;
}
```

Resultado:

```
+-----+-----+
|           |           |
|   Item 1   |   Item 2   |
|           |           |
+-----+-----+
```

9. Criando linhas com grid-template-rows

Assim como podemos definir colunas, podemos definir linhas.

A propriedade:

```
grid-template-rows
```

define o tamanho das linhas.

Exemplo:

```
.container {
  display: grid;

  /* Duas colunas */
  grid-template-columns: 1fr 1fr;

  /* Duas linhas */
  grid-template-rows: 100px 200px;
}
```

Teremos:

```
+-----+-----+
|           |           |
|   100px    |   100px    |
|           |           |
+-----+-----+
|           |           |
|   200px    |   200px    |
|           |           |
+-----+-----+
```

10. Usando auto

Podemos utilizar:

```
grid-template-rows: auto;
```

O valor auto permite que o navegador determine o tamanho necessário de acordo com o conteúdo e o espaço disponível.

Exemplo:

```
.container {  
  display: grid;  
  
  grid-template-columns: 1fr 1fr;  
  
  /* A altura das linhas será determinada automaticamente */  
  grid-template-rows: auto auto;  
}
```

11. Criando várias colunas com repeat()

Quando precisamos repetir o mesmo valor, podemos utilizar:

```
repeat()
```

Em vez de escrever:

```
grid-template-columns: 1fr 1fr 1fr;
```

podemos escrever:

```
grid-template-columns: repeat(3, 1fr);
```

O significado é:

```
repeat(quantidade, tamanho)
```

Portanto:

```
repeat(3, 1fr)
```

significa:

Crie 3 colunas de 1fr.

12. Exemplo com repeat()

```
.container {  
  display: grid;  
  
  /* Cria 4 colunas iguais */  
  grid-template-columns: repeat(4, 1fr);  
}
```

Resultado:

```
+-----+-----+-----+  
|  1   |  2   |  3   |  4   |  
+-----+-----+-----+
```

13. gap

A propriedade:

Gap

define o espaço entre os elementos do Grid.

Exemplo:

```
.container {  
  display: grid;  
  
  grid-template-columns: repeat(3, 1fr);  
  
  /* Espaço de 20px entre linhas e colunas */  
  gap: 20px;  
}
```

Visualmente:

```
+-----+   +-----+   +-----+  
|       |   |       |   |       |  
| Item 1 |   | Item 2 |   | Item 3 |  
|       |   |       |   |       |  
+-----+   +-----+   +-----+
```

14. column-gap

Podemos controlar somente o espaço entre as colunas.

```
.container {  
  column-gap: 30px;  
}
```

Nesse caso, será aplicado um espaço horizontal de 30px entre as colunas.

15. row-gap

Também podemos controlar somente o espaço entre as linhas.

```
.container {  
  row-gap: 20px;  
}
```

Será criado um espaço vertical de 20px entre as linhas.

16. Diferença entre gap, row-gap e column-gap

Podemos resumir:

gap: 20px;

Aplica espaçamento entre linhas e colunas.

row-gap: 20px;

Aplica espaçamento entre linhas.

```
column-gap: 20px;
```

Aplica espaçamento entre colunas.

Também podemos escrever:

```
gap: 20px 30px;
```

Nesse caso:

20px → linhas

30px → colunas

17. Exemplo completo de Grid

HTML:

```
<section class="produtos">
  <article class="produto">
    Produto 1
  </article>
  <article class="produto">
    Produto 2
  </article>
  <article class="produto">
    Produto 3
  </article>
  <article class="produto">
    Produto 4
  </article>
  <article class="produto">
    Produto 5
  </article>
  <article class="produto">
    Produto 6
  </article>
</section>
```

CSS:

```
.produtos {
  /* Ativa o Grid */
  display: grid;
  /* Cria três colunas iguais */
  grid-template-columns: repeat(3, 1fr);
  /* Espaçamento entre os elementos */
  gap: 20px;
}

.produto {
  padding: 30px;
  /* Apenas para visualização */
  background-color: lightgray;
  /* Centraliza o texto */
  text-align: center;
}
```

Resultado:

```
+-----+-----+-----+
| Produto 1| Produto 2| Produto 3|
+-----+-----+-----+
| Produto 4| Produto 5| Produto 6|
+-----+-----+-----+
```

18. Posicionando elementos com grid-column

Podemos fazer um item ocupar mais de uma coluna utilizando:

grid-column

Exemplo:

```
.item1 {
  /* Ocupa duas colunas */
  grid-column: span 2;
}
```

Resultado:

```
+-----+-----+
|          |          |
|      Item 1      |      Item 2      |
|          |          |
+-----+-----+
| Item 3 | Item 4 | Item 5 |
+-----+-----+
```

19. grid-column-start e grid-column-end

Também podemos determinar exatamente onde o elemento começa e termina.

Exemplo:

```
.item {
  grid-column-start: 1;
  grid-column-end: 3;
}
```

O elemento começará na linha da coluna 1 e terminará na linha da coluna 3.

Isso significa que ele ocupará duas colunas.

20. grid-column: 1 / 3

Podemos simplificar:

```
.item {
  grid-column: 1 / 3;
}
```

Isso equivale a:

```
.item {  
  grid-column-start: 1;  
  grid-column-end: 3;  
}
```

21. grid-row

A propriedade:

`grid-row`

permite controlar a posição vertical de um elemento.

Exemplo:

```
.item {  
  grid-row: 1 / 3;  
}
```

O elemento ocupará duas linhas.

22. grid-row: span

Também podemos utilizar:

```
.item {  
  grid-row: span 2;  
}
```

Isso significa:

O elemento ocupará duas linhas.

Da mesma forma:

```
.item {  
  grid-column: span 2;  
}
```

significa:

O elemento ocupará duas colunas.

23. Exemplo prático de grid-column e grid-row

HTML:

```
<div class="layout">  
  <header>Header</header>  
  <aside>Menu</aside>  
  <main>Conteúdo</main>  
  <footer>Footer</footer>  
</div>
```

CSS:

```
.layout {
  display: grid;
  /* Três linhas */
  grid-template-rows: 80px 1fr 60px;
  /* Duas colunas */
  grid-template-columns: 200px 1fr;
  gap: 10px;
}
header {
  /* Header ocupa as duas colunas */
  grid-column: 1 / 3;
}
aside {
  /* Menu ocupa a segunda área */
}
main {
  /* Conteúdo principal */
}
footer {
  /* Footer ocupa as duas colunas */
  grid-column: 1 / 3;
}
```

Visualmente:

```
+-----+
|                HEADER                |
+-----+-----+
|  MENU  |  CONTEÚDO  |
|                |
+-----+-----+
|                FOOTER                |
+-----+
```

24. grid-template-areas

Uma das formas mais interessantes de organizar layouts é utilizando:

grid-template-areas

Podemos atribuir nomes às regiões do Grid.

Exemplo:

```
.layout {
  display: grid;
  grid-template-columns: 200px 1fr;
  grid-template-rows: 80px 1fr 60px;
  grid-template-areas:
    "header header"
    "menu   conteudo"
    "footer footer";
}
```

Agora podemos indicar qual elemento pertence a cada área.

```
header {
    grid-area: header;
}
aside {
    grid-area: menu;
}
main {
    grid-area: conteudo;
}
footer {
    grid-area: footer;
}
```

25. Exemplo completo com grid-template-areas

HTML:

```
<div class="layout">
  <header>
    Cabeçalho
  </header>
  <aside>
    Menu
  </aside>
  <main>
    Conteúdo principal
  </main>
  <footer>
    Rodapé
  </footer>
</div>
```

CSS:

```
.layout {
    /* Ativa o Grid */
    display: grid;
    /* Define duas colunas */
    grid-template-columns: 200px 1fr;
    /* Define três linhas */
    grid-template-rows: 80px 1fr 60px;
    /* Define o desenho do layout */
    grid-template-areas:
        "header header"
        "menu conteudo"
        "footer footer";
    /* Espaço entre as áreas */
    gap: 10px;
}
/* Define a área do cabeçalho */
header {
    grid-area: header;
}
/* Define a área do menu */
aside {
    grid-area: menu;
}
/* Define a área do conteúdo */
```

```
main {
    grid-area: conteudo;
}
/* Define a área do rodapé */
footer {
    grid-area: footer;
}
```

Visualmente:

```
+-----+
|                HEADER                |
+-----+-----+
|      MENU      |      CONTEÚDO      |
|                |                    |
+-----+-----+
|                FOOTER                |
+-----+
```

Essa abordagem torna o código bastante fácil de compreender.

26. justify-items

A propriedade:

`justify-items`

controla o alinhamento dos itens no eixo horizontal dentro de suas células.

Valores comuns:

```
justify-items: start;
justify-items: center;
justify-items: end;
justify-items: stretch;
```

Exemplo:

```
.container {
    display: grid;

    grid-template-columns: repeat(3, 1fr);

    /* Centraliza os itens horizontalmente */
    justify-items: center;
}
```

27. align-items

Controla o alinhamento dos itens no eixo vertical.

Exemplo:

```
.container {
    display: grid;

    /* Centraliza verticalmente */
}
```

```
    align-items: center;
}
```

Valores comuns:

```
align-items: start;
align-items: center;
align-items: end;
align-items: stretch;
```

28. place-items

Podemos combinar:

justify-items

e:

align-items

utilizando:

place-items

Exemplo:

```
.container {
    display: grid;
    /* Centraliza horizontal e verticalmente */
    place-items: center;
}
```

É equivalente a:

```
.container {
    justify-items: center;
    align-items: center;
}
```

29. justify-content

Essa propriedade controla o alinhamento do **Grid inteiro** dentro do container.

Exemplo:

```
.container {
    display: grid;

    grid-template-columns:
        200px 200px;

    /* Centraliza o Grid horizontalmente */
    justify-content: center;
}
```

Valores comuns:

```
justify-content: start;
justify-content: center;
justify-content: end;
justify-content: space-between;
justify-content: space-around;
justify-content: space-evenly;
```

30. align-content

Controla o alinhamento do Grid inteiro no eixo vertical.

Exemplo:

```
.container {
  display: grid;

  align-content: center;
}
```

Valores comuns:

```
align-content: start;
align-content: center;
align-content: end;
align-content: space-between;
align-content: space-around;
align-content: space-evenly;
```

31. place-content

Combina:

justify-content

e:

align-content

Exemplo:

```
.container {
  display: grid;

  /* Centraliza o Grid horizontal e verticalmente */
  place-content: center;
}
```

32. justify-self

Permite controlar o alinhamento de **um item específico** no eixo horizontal.

Exemplo:

```
.item1 {
  justify-self: center;
}
```

Enquanto os outros itens permanecem com o comportamento padrão.

Valores:

```
justify-self: start;  
justify-self: center;  
justify-self: end;  
justify-self: stretch;
```

33. align-self

Controla o alinhamento de um item específico no eixo vertical.

Exemplo:

```
.item1 {  
    align-self: center;  
}
```

Valores:

```
align-self: start;  
align-self: center;  
align-self: end;  
align-self: stretch;
```

34. place-self

Combina:

justify-self

e:

align-self

Exemplo:

```
.item1 {  
    /* Centraliza o item horizontal e verticalmente */  
    place-self: center;  
}
```

35. minmax()

A função:

minmax()

é extremamente importante para criar Grid responsivo.

Sintaxe:

minmax(mínimo, máximo)

Exemplo:

```
grid-template-columns:
  minmax(200px, 1fr);
```

Isso significa:

A coluna terá no mínimo 200px e poderá crescer até ocupar uma fração do espaço disponível.

36. auto-fit

O valor:

```
auto-fit
```

permite ajustar automaticamente a quantidade de colunas que cabe no espaço disponível.

Exemplo:

```
.container {
  display: grid;
  grid-template-columns:
    repeat(auto-fit, minmax(250px, 1fr));
  gap: 20px;
}
```

Em uma tela grande:

```
+-----+ +-----+ +-----+ +-----+
| Card 1 | | Card 2 | | Card 3 | | Card 4 |
+-----+ +-----+ +-----+ +-----+
```

Em uma tela menor:

```
+-----+ +-----+
| Card 1 | | Card 2 |
+-----+ +-----+

+-----+ +-----+
| Card 3 | | Card 4 |
+-----+ +-----+
```

Em uma tela muito pequena:

```
+-----+
| Card 1 |
+-----+

+-----+
| Card 2 |
+-----+

+-----+
| Card 3 |
+-----+
```

37. auto-fill

Também podemos utilizar:

auto-fill

Exemplo:

```
.container {
  display: grid;
  grid-template-columns:
    repeat(auto-fill, minmax(250px, 1fr));
  gap: 20px;
}
```

auto-fill e auto-fit possuem comportamentos semelhantes, mas podem apresentar diferenças quando existe espaço disponível para colunas vazias.

Para os primeiros exemplos de responsividade, auto-fit costuma ser mais intuitivo.

38. Exemplo de Grid Responsivo

HTML:

```
<section class="cards">
  <article class="card">
    <h2>HTML</h2>
    <p>Estrutura da página.</p>
  </article>
  <article class="card">
    <h2>CSS</h2>
    <p>Estilização da página.</p>
  </article>
  <article class="card">
    <h2>JavaScript</h2>
    <p>Comportamento da página.</p>
  </article>
  <article class="card">
    <h2>React</h2>
    <p>Biblioteca para interfaces.</p>
  </article>
</section>
```

CSS:

```
.cards {
  /* Ativa o Grid */
  display: grid;
  /*
    Cria automaticamente a quantidade
    de colunas que couber no espaço.
  */
  grid-template-columns:
    repeat(
      auto-fit,
      minmax(250px, 1fr)
    );
  /* Espaço entre os cards */
  gap: 20px;
}
```

```

}
.card {
  /* Espaçamento interno */
  padding: 20px;
  /* Apenas para visualização */
  border: 1px solid #ccc;
  /* Evita que o conteúdo fique apertado */
  min-height: 150px;
}

```

Esse é um dos padrões mais úteis para criar galerias e conjuntos de cards responsivos.

39. Grid e Media Queries

Embora seja possível criar muitos layouts responsivos sem Media Queries, elas continuam sendo importantes.

Exemplo:

```

.container {
  display: grid;
  /* Mobile */
  grid-template-columns: 1fr;
}
@media (min-width: 768px) {
  .container {
    /* Tablet */
    grid-template-columns: 1fr 1fr;
  }
}
@media (min-width: 1024px) {
  .container {
    /* Desktop */
    grid-template-columns:
      repeat(3, 1fr);
  }
}

```

Resultado:

CELULAR

```

+-----+
|    1    |
+-----+
|    2    |
+-----+
|    3    |
+-----+

```

Tablet:

```

+-----+-----+
|    1    |    2    |
+-----+-----+
|    3    |
+-----+

```

Desktop:

1	2	3	

40. Grid x Flexbox

É importante que o estudante saiba quando utilizar cada tecnologia.

Flexbox

É especialmente adequado para organizar elementos em uma dimensão.

Exemplo:

linha

```
[ Item ] [ Item ] [ Item ]
```

ou:

coluna

```
[ Item ]  
[ Item ]  
[ Item ]
```

Exemplo:

```
.menu {  
  display: flex;  
  gap: 20px;  
}
```

Grid

É especialmente adequado quando precisamos controlar linhas e colunas.

Exemplo:

```
[ 1 ][ 2 ][ 3 ]  
[ 4 ][ 5 ][ 6 ]  
[ 7 ][ 8 ][ 9 ]
```

Exemplo:

```
.galeria {  
  display: grid;  
  grid-template-columns:  
    repeat(3, 1fr);  
}
```

Uma regra prática:

Flexbox é excelente para organizar elementos em uma dimensão. Grid é excelente para organizar layouts em duas dimensões.

Não significa que um seja substituto do outro. É comum utilizar os dois no mesmo projeto.

41. Exemplo de página completa utilizando Grid

HTML:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>Exemplo com CSS Grid</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="layout">
    <header>Cabeçalho</header>
    <nav>
      Menu
    </nav>
    <main>
      <h1>Conteúdo</h1>
      <section class="cards">
        <article>
          Card 1
        </article>
        <article>
          Card 2
        </article>
        <article>
          Card 3
        </article>
      </section>
    </main>
    <footer>
      Rodapé
    </footer>
  </div>
</body>
</html>
```

CSS:

```
* {
  /* Inclui padding e border dentro
    do tamanho definido */
  box-sizing: border-box;
}
body {
  /* Remove margem padrão */
  margin: 0;
  /* Define uma fonte */
  font-family: Arial, sans-serif;
}
.layout {
  /* Ativa o Grid */
  display: grid;
  /* Define duas colunas */
  grid-template-columns:
    200px 1fr;
```

```

    /* Define três linhas */
    grid-template-rows:
        80px 1fr 60px;
    /* Define as áreas */
    grid-template-areas:
        "header header"
        "menu conteudo"
        "footer footer";
    /* Espaçamento */
    gap: 10px;
    /* Altura mínima */
    min-height: 100vh;
}
header {
    /* Posiciona o elemento na área header */
    grid-area: header;
    padding: 20px;
    background-color: #ddd;
}
nav {
    /* Posiciona o menu */
    grid-area: menu;
    padding: 20px;
    background-color: #eee;
}
main {
    /* Posiciona o conteúdo */
    grid-area: conteudo;
    padding: 20px;
}
footer {
    /* Posiciona o rodapé */
    grid-area: footer;
    padding: 20px;
    background-color: #ddd;
}
.cards {
    /* Ativa Grid para os cards */
    display: grid;
    /*
        Cria automaticamente as colunas
        conforme o espaço disponível.
    */
    grid-template-columns:
        repeat(
            auto-fit,
            minmax(200px, 1fr)
        );

    gap: 20px;
}
.cards article {
    padding: 30px;
    border: 1px solid #ccc;
    text-align: center;
}
/* Responsividade */
@media (max-width: 700px) {
    .layout {
        /*
            Em telas pequenas,

```

```
        o menu passa para cima
        do conteúdo.
    */
    grid-template-columns: 1fr;
    grid-template-rows:
        auto
        auto
        1fr
        auto;
    grid-template-areas:
        "header"
        "menu"
        "conteudo"
        "footer";
}
}
```

42. Principais propriedades do CSS Grid

Propriedade	Função
display: grid	Ativa o Grid
grid-template-columns	Define as colunas
grid-template-rows	Define as linhas
grid-template-areas	Define áreas nomeadas
grid-column	Define posição/ocupação horizontal
grid-row	Define posição/ocupação vertical
grid-column-start	Define onde o item começa horizontalmente
grid-column-end	Define onde o item termina horizontalmente
grid-row-start	Define onde o item começa verticalmente
grid-row-end	Define onde o item termina verticalmente
grid-area	Define uma área ou posição
gap	Define espaço entre linhas e colunas
row-gap	Define espaço entre linhas
column-gap	Define espaço entre colunas
justify-items	Alinha itens horizontalmente
align-items	Alinha itens verticalmente
place-items	Combina justify-items e align-items
justify-content	Alinha o Grid horizontalmente
align-content	Alinha o Grid verticalmente
place-content	Combina justify-content e align-content
justify-self	Alinha um item horizontalmente
align-self	Alinha um item verticalmente
place-self	Combina justify-self e align-self

43. Principais funções e valores

repeat()

Repete colunas ou linhas.

```
grid-template-columns:  
  repeat(3, 1fr);
```

minmax()

Define limite mínimo e máximo.

```
grid-template-columns:  
  minmax(200px, 1fr);
```

auto-fit

Ajusta automaticamente a quantidade de colunas.

```
grid-template-columns:  
  repeat(auto-fit, minmax(250px, 1fr));
```

auto-fill

Preenche o espaço disponível considerando as possíveis faixas do Grid.

```
grid-template-columns:  
  repeat(auto-fill, minmax(250px, 1fr));
```

span

Faz um item ocupar várias linhas ou colunas.

```
grid-column: span 2;
```

ou:

```
grid-row: span 2;
```

fr

Divide o espaço disponível em frações.

```
grid-template-columns:  
  1fr 2fr 1fr;
```

44. Exercício Prático 1 — Galeria

Crie uma galeria com seis imagens.

HTML:

```
<section class="galeria">
  
  
  
  
  
  
</section>
```

CSS:

```
.galeria {
  /* Ativa o Grid */
  display: grid;
  /* Grid responsivo */
  grid-template-columns:
    repeat(
      auto-fit,
      minmax(200px, 1fr)
    );
  /* Espaçamento */
  gap: 15px;
}
.galeria img {
  /* Imagem nunca ultrapassa o container */
  max-width: 100%;
  /* Mantém a proporção */
  height: auto;
}
```

45. Exercício Prático 2 — Cards

Crie seis cards:

```
<section class="cards">
  <article>Card 1</article>
  <article>Card 2</article>
  <article>Card 3</article>
  <article>Card 4</article>
  <article>Card 5</article>
  <article>Card 6</article>
</section>
```

Utilize:

```
.cards {
  display: grid;
  grid-template-columns:
    repeat(
      auto-fit,
      minmax(250px, 1fr)
    );
  gap: 20px;
}
```

Teste o projeto em diferentes larguras.

46. Exercício Prático 3 — Layout de Site

Crie o seguinte layout:

HEADER	
MENU	CONTEÚDO
FOOTER	

Utilize:

```
grid-template-areas:  
  "header header"  
  "menu conteudo"  
  "footer footer";
```

Depois crie uma Media Query para transformar o layout em uma única coluna em telas pequenas.

47. Resumo

O CSS Grid é um sistema de layout baseado em linhas e colunas.

A propriedade principal é:

```
display: grid;
```

As propriedades mais importantes para começar são:

```
grid-template-columns  
grid-template-rows  
gap
```

Depois podemos avançar para:

```
grid-column  
grid-row  
grid-area  
grid-template-areas
```

E finalmente utilizar recursos avançados para responsividade:

```
repeat()  
minmax()  
auto-fit  
auto-fill  
fr
```

Um dos padrões mais importantes para memorizar é:

```
.container {  
  display: grid;  
  grid-template-columns:  
    repeat(  
      auto-fit,  
      minmax(250px, 1fr)  
    );  
  gap: 20px;  
}
```

Esse padrão permite criar uma estrutura de Grid que se adapta automaticamente ao espaço disponível.

Conceito final

Podemos resumir o CSS Grid da seguinte maneira:

```
GRID  
|  
+-- Container  
|  
+-- Linhas  
|  
+-- Colunas  
|  
+-- Grid Items  
|  
+-- Espaçamento  
|  
+-- Posicionamento  
|  
+-- Áreas  
|  
+-- Responsividade
```

A principal ideia que o estudante deve compreender é:

CSS Grid permite organizar elementos em linhas e colunas, controlar suas dimensões, posicioná-los em diferentes áreas e criar layouts que se adaptam ao espaço disponível.

Quando combinado com Flexbox, Media Queries e unidades relativas, o CSS Grid se torna uma das principais ferramentas para construção de interfaces Web responsivas.